

Machine Learning-Enhanced Firewall Attack Detection

A Hybrid Architecture for Early Warning and Threat Enrichment

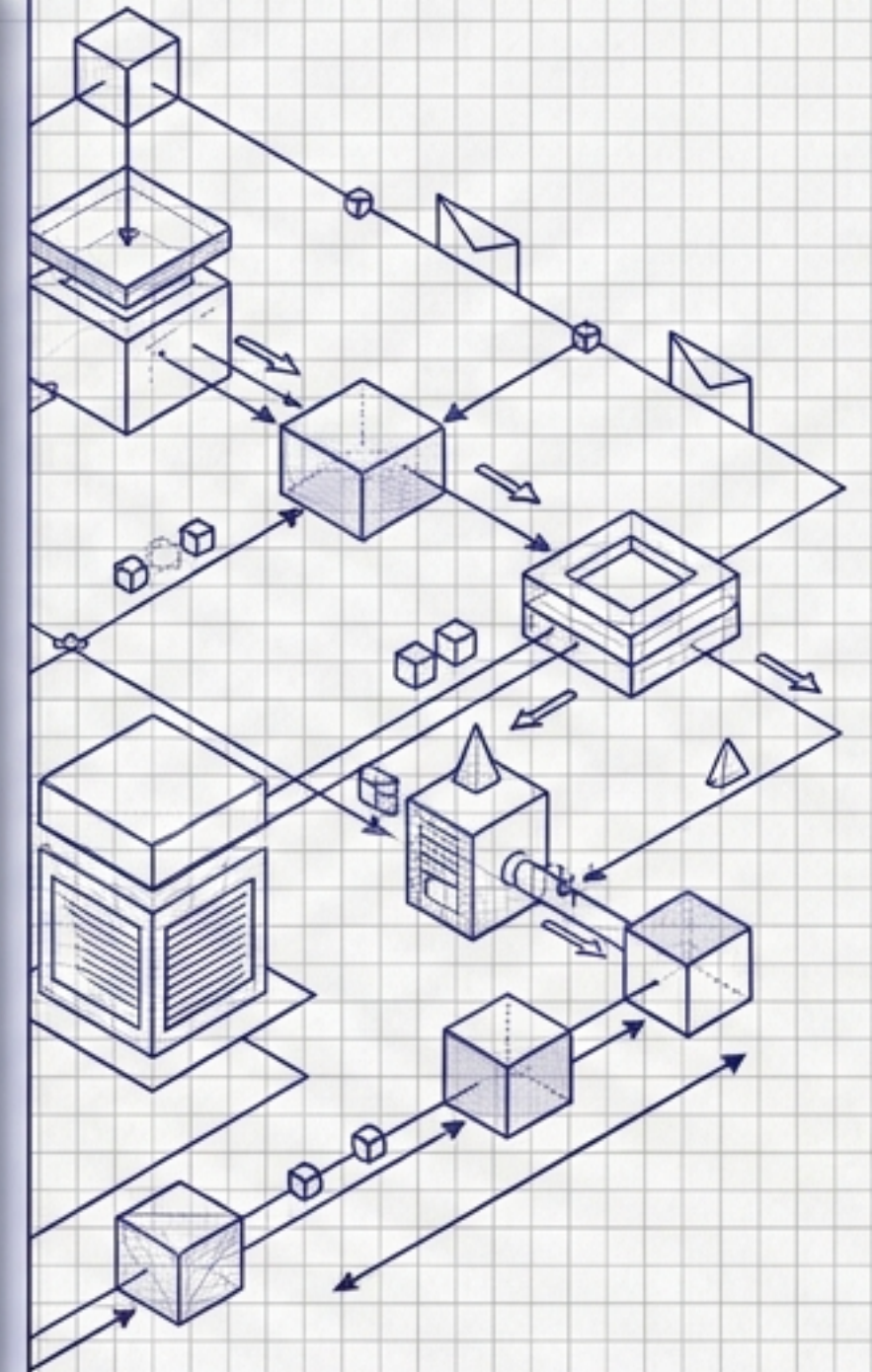
[Final Year Project]

[Python]

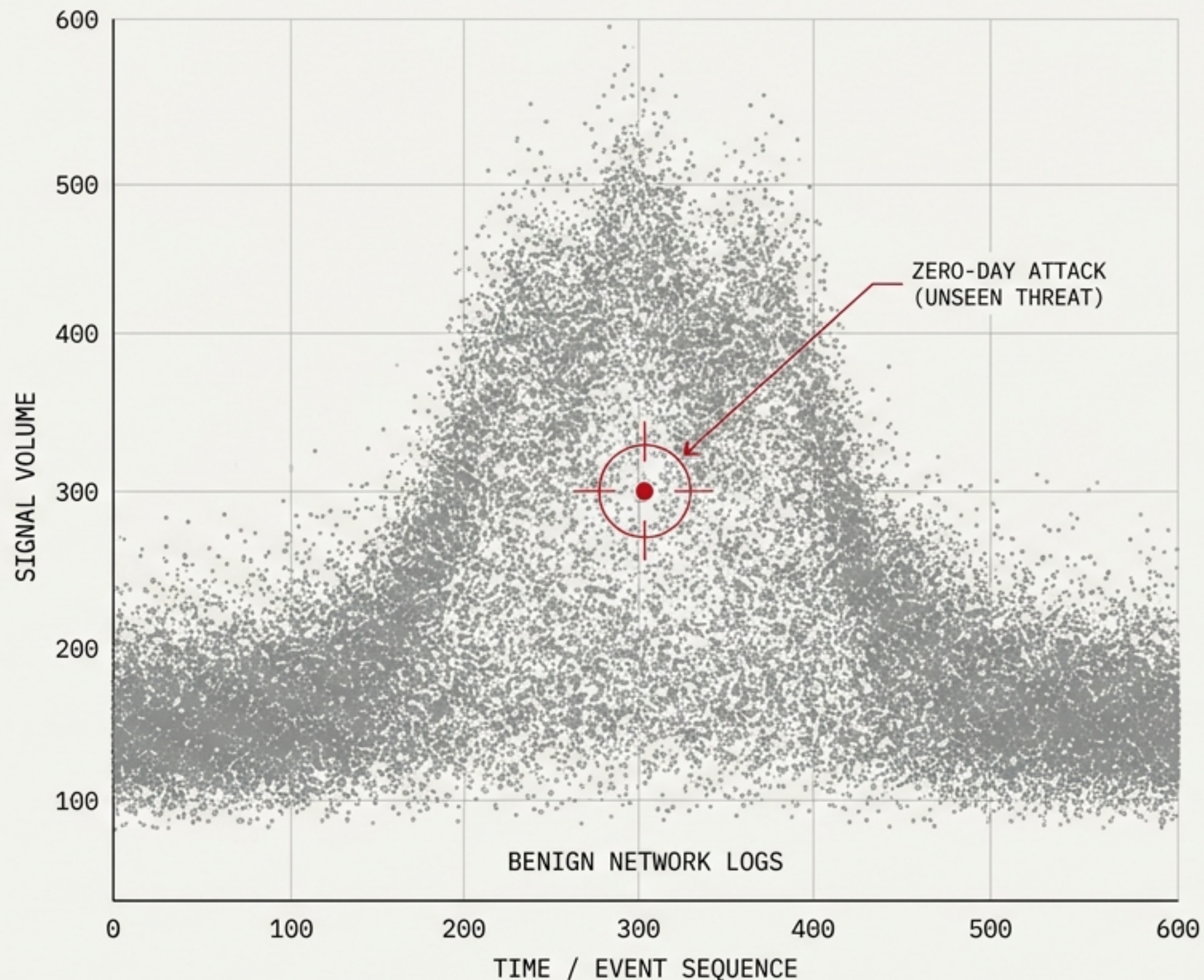
[RandomForest]

[Cyber Defence]

Sombit Pramanik



The Core Vulnerability in Network Defence



Blind to the Novel

Traditional firewalls rely on static signatures. They detect only known threats and fail against evolving, zero-day attack patterns.

Analyst Alert Fatigue

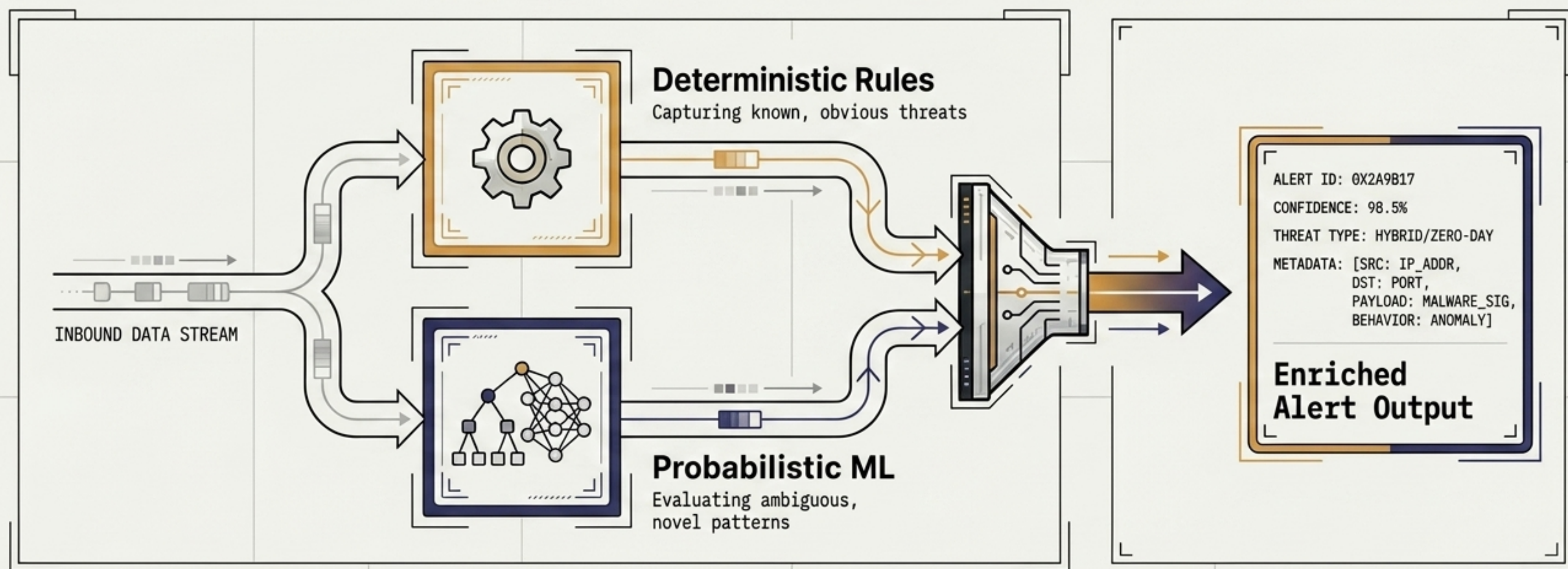
The sheer volume of raw, unstructured logs buries security operations centres in noise, making manual analysis impossible.

The Consequence

Unacceptable rates of false positives and false negatives.

The Hybrid Solution Paradigm

We enhance, rather than replace, traditional firewalls. By merging simple pattern heuristics for immediate early warning with a machine learning classification engine for novel threat detection, the system generates enriched, high-confidence alerts with precise metadata.

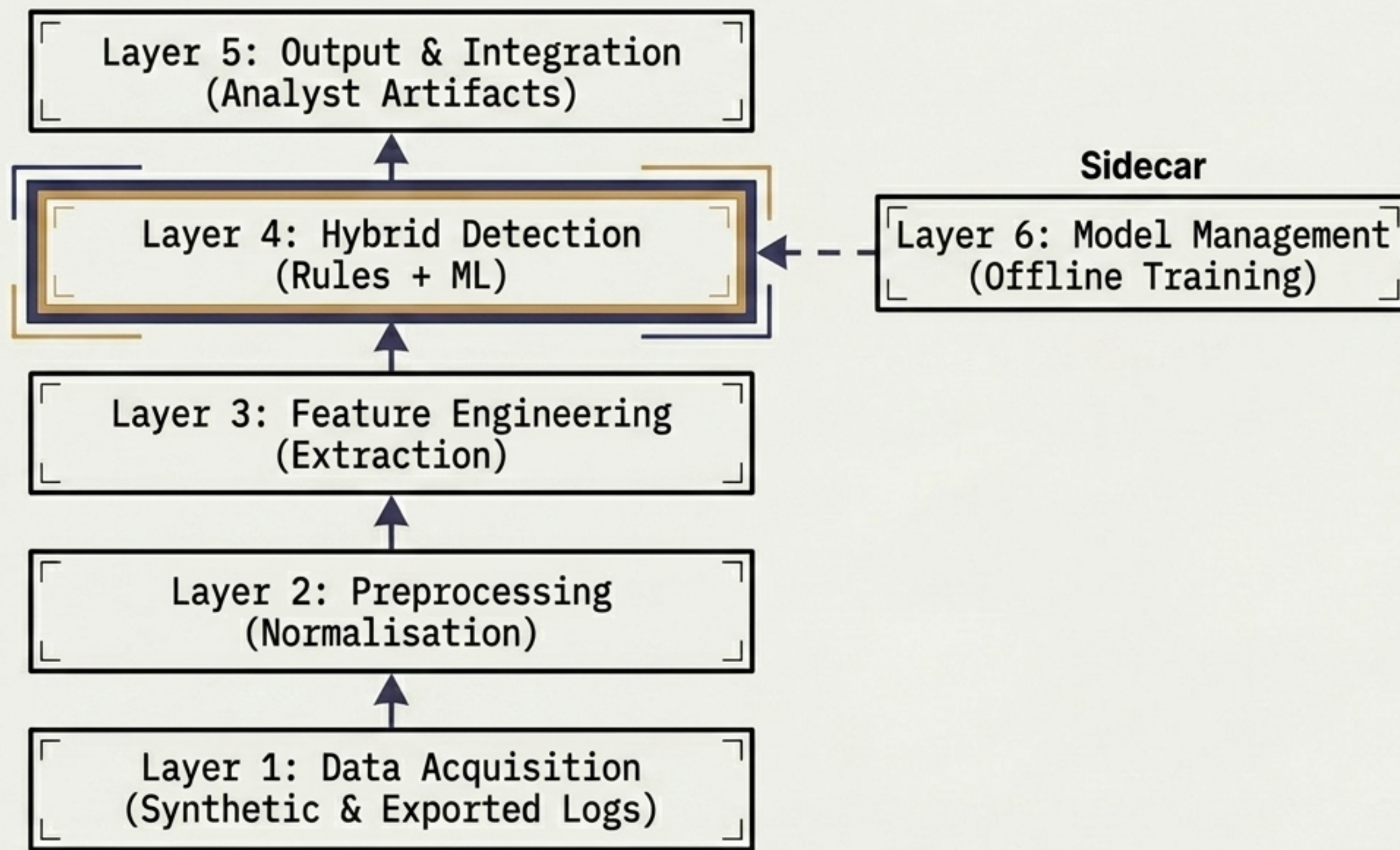


Legacy vs. Hybrid Architecture

Operational Dimension	Legacy Firewalls	Hybrid ML-Enhanced Architecture
Detection Mechanism	Static Signatures	Deterministic Rules + Probabilistic ML
Threat Scope	Known Signatures Only	Known Patterns + Novel / Zero-Day Behaviours
Analyst Output	Raw, Unstructured Logs	Enriched Alerts with Confidence Scores
System Adaptability	Manual Signature Updates	Capable of Continuous Model Learning

The 6-Layer Modular Architecture

Unidirectional flow ensures offline
model training is strictly separated
from online detection runtimes.



Layers 1 & 2: Safe Development via Synthetic Generation

```
> python synthetic_firewall_logs.py --intensity 3 --live
```

```
[INFO] Initialising log stream... Timezone: UTC
192.168.1.104 -> 10.0.0.5 : 443 [ALLOW]
192.168.1.201 -> 10.0.0.5 : 80 [ALLOW]
[INJECT] Initiating Port Scan pattern from 203.0.113.42...
203.0.113.42 -> 10.0.0.8 : 22 [DENY]
203.0.113.42 -> 10.0.0.8 : 23 [DENY]
203.0.113.42 -> 10.0.0.8 : 8080 [DENY]
[INJECT] Initiating DoS pattern...
```

The `synthetic_firewall_logs.py` module solves the critical privacy and data scarcity problem in **cybersecurity research**. It generates **realistic, labelled data**—safe from PII exposure—with configurable attack injection intensities to simulate high-stress network environments.

Layer 3: Feature Engineering & Transformation

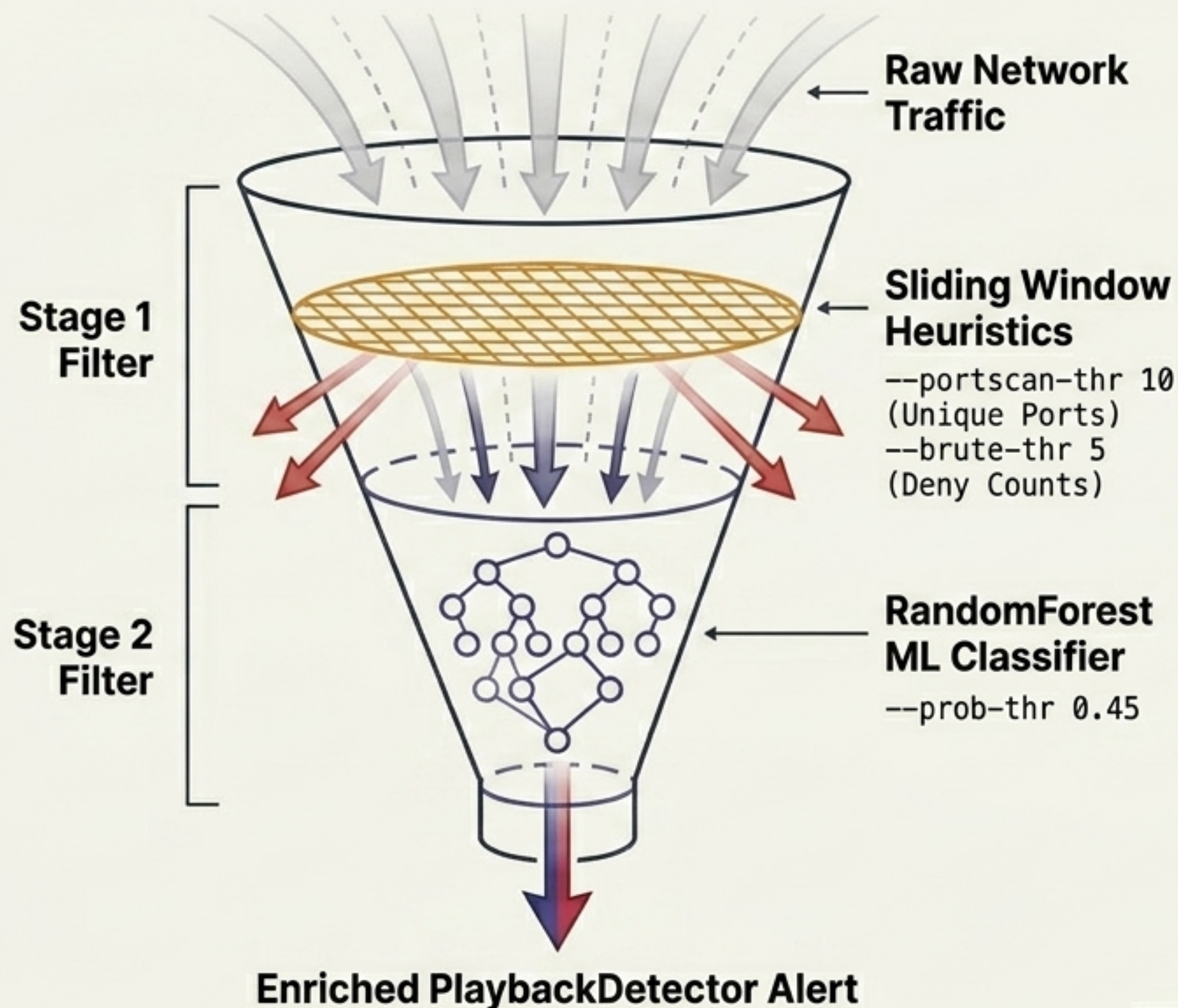
The parsing pipeline standardises raw variables and extracts behavioural indicators over time-windows, outputting a canonical CSV dataset ready for ML ingestion.



Layer 4: The Hybrid Detection Runtime

Deterministic rules catch obvious port scans and brute force attacks instantly with minimal compute.

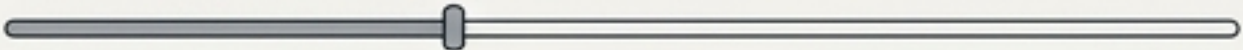
The ML model handles the complex, obfuscated anomalies, assigning a precise attack probability.



Layer 5: The Analyst-Centric Replay UI

The system writes to detections.log with a configurable --pause flag to simulate real-time analysis. It provides side-by-side terminal views, giving analysts the exact context and confidence scores required to make rapid, informed decisions.

Original Raw Log		Enriched Output	
14:32:05 SRC: 203.0.113.42 DST: 10.0.0.8 DPORT: 22 PROTO: TCP ACT: DENY	1	[dst_port_entropy: 0.95]	
	2	Rule Match: Port Scan	
	3	ML Probability: 0.85	
	4	THREAT TAG: ATTACK_CONFIRMED	



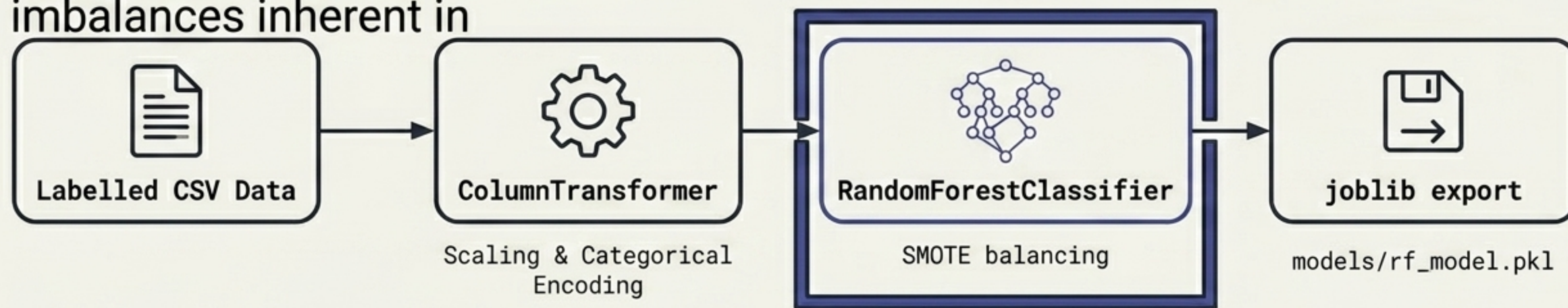
--pause 1.5s

Layer 6: Model Training & Evaluation

Built on Scikit-Learn and Pandas.

The `train_model()` function builds a complete preprocessing and classification pipeline.

Employs techniques like SMOTE (Synthetic Minority Over-sampling Technique) to handle severe class imbalances inherent in



Current Implementation Status

What executes in the repository today.



Configurable Synthetic Log Generator

`synthetic_firewall_logs.py`



Automated CSV Dataset Builder

`build_dataset_from_log`



RandomForest Training Pipeline

`train_model`



Hybrid Detection Runtime (Rules + ML)

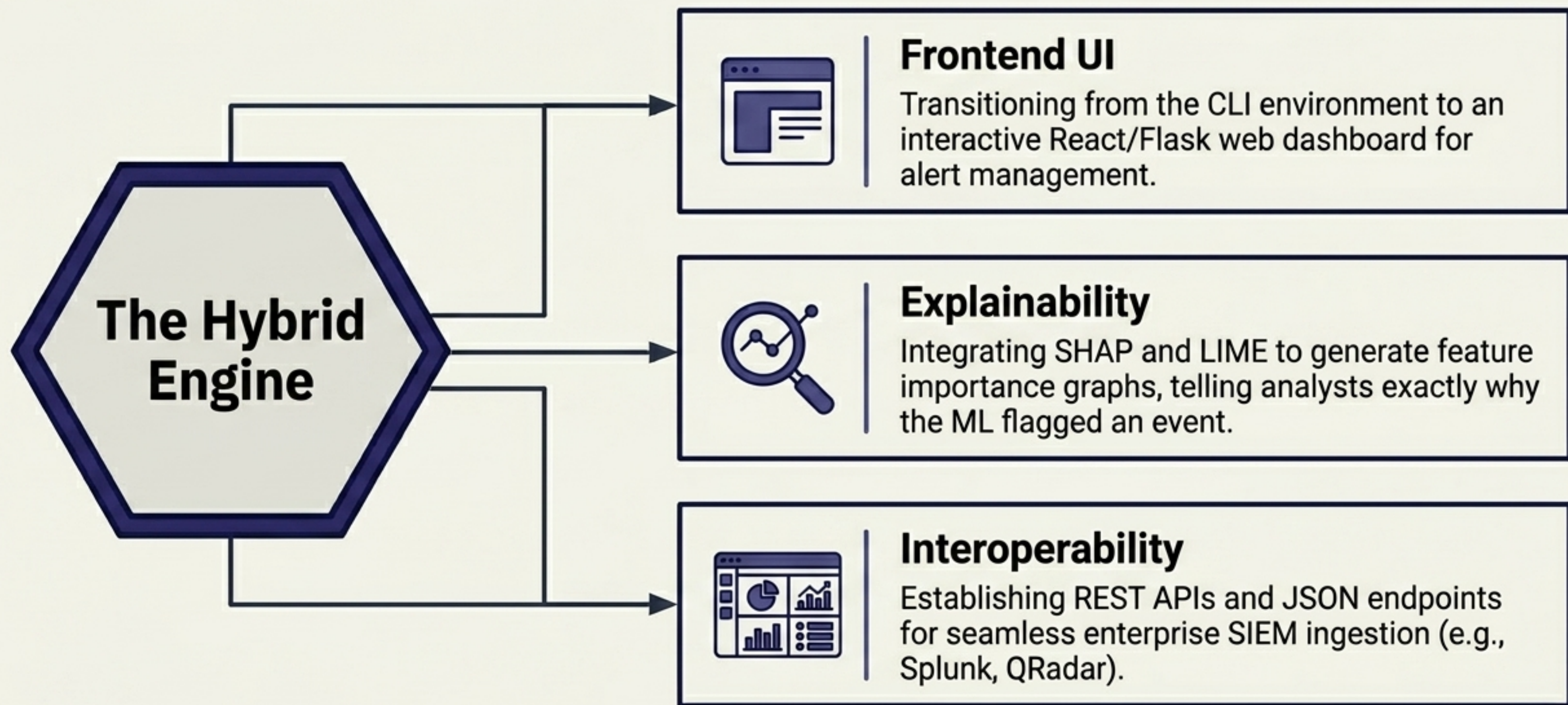
`PlaybackDetector`



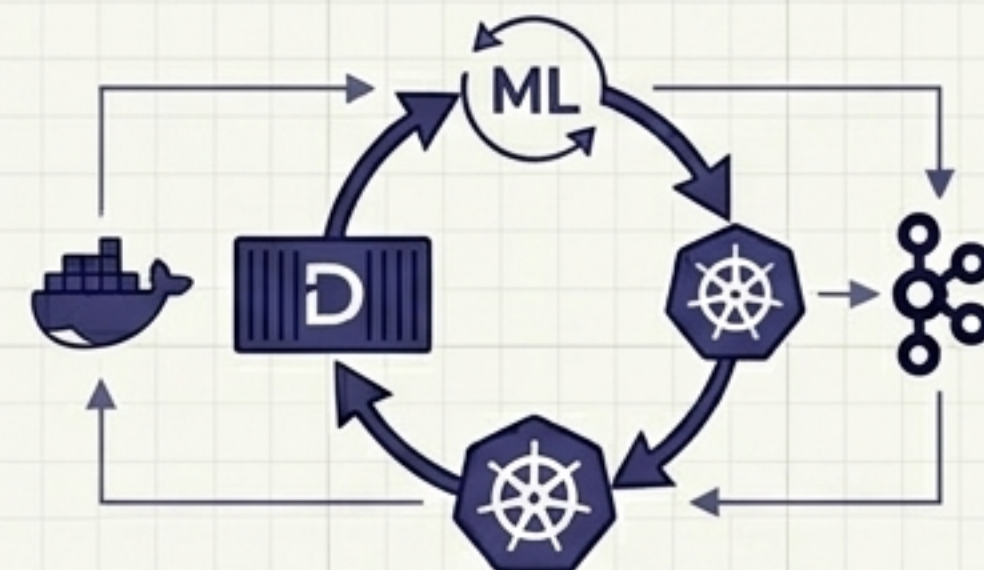
Side-by-Side Replay UI & CLI Orchestration

`--pause, --speedup`

Phase 2 Roadmap: Integration & Explainability



Phase 3 Roadmap: Advanced MLOps & Scale



Adaptation



Implementing online and incremental learning to automatically counter concept drift and adversarial evasion tactics without manual retraining.

Scale



Transitioning to streaming data architectures (**Kafka**) and containerised microservices (**Docker/Kubernetes**) for robust, high-volume production deployment.

Context



Expanding multi-source correlation by ingesting parallel telemetry from external **Intrusion Detection Systems (IDS)** and Active Directory logs.

Core Strengths & Value Proposition



Modular Extensibility

Built as independent components. The architecture is instantly ready for new parsers, feature extractors, or ensemble models like XGBoost.



Hybrid Performance

Merges deterministic reliability with probabilistic adaptability, drastically lowering operational costs and false positive alert fatigue.



Analyst-First Design

Prioritises enriched situational awareness and confidence metadata over opaque, black-box automated blocking.


```
root@soc-eval:~$ python ml_firewall_system.py --demo
```

Open for architecture review, codebase inspection, or live demonstration of the Synthetic Log Generator.

GitHub: [SombitPramanik/ML_Base_Attack_Detection_System_on_Firewall](https://github.com/SombitPramanik/ML_Base_Attack_Detection_System_on_Firewall)